AutoHeal AI: Complete Technical Specification

End-to-End Pipeline: Ingestion → Preprocessing → 7-Pass Healer → Automated EDA → Bias-Free Feature Engineering → Dashboard

1. System Architecture & Lifecycle

AutoHeal AI is an autonomous, production-grade data pipeline that transforms raw, corrupted enterprise tables into statistically clean, fully explored, and ML-ready feature matrices with zero human intervention.

1.1 End-to-End Pipeline Stages



2. Preprocessing & Self-Healing Enhancements

2.1 Pass 1: Header Collision Disambiguation

When datasets originate from human data entry or distributed databases, column headers often contain subtle differences that collapse into duplicates upon normalization (e.g., `"User Name"` and `"User_Name"` both becoming `"user_name"`). In standard Pandas, duplicate column names cause `df['user_name']` to return a DataFrame instead of a Series, crashing all downstream transformations.

Implemented Solution (detector/healer.py): The header stabilizer tracks normalized column names using a hash dictionary `seen = {}`. If a normalized header has already been registered, it appends a monotonic suffix (e.g., `user_name`, `user_name_2`, `user_name_3`). This guarantees unique schema identification while preserving clean `snake_case` formatting.

2.2 Pass 2.5: High-Missingness Column Removal

Standard imputation engines blindly impute missing values even when 95% or 99% of the feature is null, manufacturing fictitious data that degrades predictive model accuracy.

Implemented Solution (detector/healer.py): Following sentinel extraction, the pipeline evaluates the missingness ratio: $$r_{ ext{missing}} = rac{N_{ ext{NaN}}}{N_{ ext{rows}}}$$ If $r_{ ext{missing}} \ge 0.90$ (the `max_missing_drop_threshold` defined in `config.py`), the column is dropped from the dataset. The drop operation is recorded in the `HealingLedger` under issue type `COLUMN_DROPPED` with full audit telemetry.

3. Automated Exploratory Data Analysis (EDA) Engine

Implemented in `detector/eda.py`, the `ExploratoryDataAnalyzer` automatically determines where and how to dispatch statistical evaluations based on feature properties:

3.1 Automated Dispatching Matrix

Feature Nature	Univariate Analysis	Bivariate Analysis	Multivariate Analysis
Continuous Numerical	Mean, Median, Std, IQR, Skewness, Kurtosis, Distribution Type	Pearson linear r, Spearman rank $ ho$ against all numerical features	Variance Inflation Factor (VIF), Collinear Clusters ($\|r\| \ge 0.85$), Eigenvalues
Categorical & Nominal	Cardinality, Mode %, Shannon Entropy $H(X)$, Imbalance Ratio	Contingency cross-tabs, Chi-Square χ^2, Cramér's V association	Cross-category dependency clustering
Numerical × Categorical	Conditional distributions (group medians and group variances)	One-Way ANOVA Eta-squared (η^2) effect size calculation	Multi-way variance decomposition
Temporal & Datetime	Timeline bounds, timestamp frequency, gap detection	Time-series correlation with numerical target metrics	Seasonality and periodic component analysis

3.2 Advanced Univariate Metrics & Distribution Classification

- **Shannon Entropy:** Measures categorical diversity:

  $$H(X) = -\sum_{i=1}^k p_i \log_2(p_i), \quad H_{ ext{normalized}} = rac{H(X)}{\log_2(k)}$$

  $H_{ ext{normalized}} pprox 1$ indicates uniform distribution across categories; $H_{ ext{normalized}} pprox 0$ indicates extreme concentration.
- **Class Imbalance Ratio:** Computes $N_{ ext{majority}} / N_{ ext{minority}}$. Values ≥ 10.0 trigger an alert for model representation bias.
- **Distribution Classification:** Evaluates Fisher-Pearson skewness G_1 and excess kurtosis a_4 to classify features into: *Gaussian / Approximately Symmetric* ($|G_1| < 0.5$, $|a_4| < 1.0$), *Right-Skewed* ($G_1 \ge 0.5$), *Left-Skewed* ($G_1 \le -0.5$), *Leptokurtic / Heavy-Tailed* ($a_4 \ge 1.5$), or *Platykurtic / Uniform* ($a_4 \le -1.0$).

3.3 Bivariate & Multivariate Statistical Formulations

- **ANOVA Eta-Squared (η^2):** Measures proportion of numerical variance explained by categorical groupings:

  $$\eta^2 = rac{SS_{ ext{between}}}{SS_{ ext{total}}} = rac{\sum n_k (ar{y}_k - ar{y})^2}{\sum (y_i - ar{y})^2}$$

  $\eta^2 \ge 0.14$ signals a large effect size where categories strongly differentiate numerical values.
- **Cramér's V Statistic:** Measures association between two categorical variables ($V \in [0, 1]$):

  $$V = \sqrt{rac{\chi^2}{N \cdot \min(r - 1, c - 1)}}$$

- **Multicollinearity Screening (VIF Proxy):** Inverts the correlation matrix R to approximate Variance Inflation Factors: $ ext{VIF}_j = (R^{-1})_{jj}$. Values ≥ 5.0 identify redundant features that destabilize model weights.

4. Bias-Mitigating Feature Engineering Engine

Implemented in `detector/features.py`, the `FeatureEngineer` addresses fundamental sources of algorithmic bias:

4.1 Variance-Stabilizing Transforms (Log1p)

- **The Bias Risk:** In linear estimators, logistic regressions, and neural networks, features with heavy positive skewness ($G_1 \ge 1.0$) allow extreme tail values to exert disproportionate leverage on loss gradients, biasing the learned parameters toward high-magnitude instances.
- **The Mathematical Solution:** Generates variance-stabilized features using the natural logarithm:

  ```
  $$x_{ ext{log1p}} = \ln(1 + x)$$
  ```

  Compresses dynamic range, normalizes gradient magnitudes, and stabilizes residual variance.

4.2 Representation Bias Mitigation (Rare-Class Consolidation)

- **The Bias Risk:** Categorical features with long-tail rare classes (frequencies $< 2.5\%$) cause tree-based models to overfit on noise and linear models to calculate unstable, extreme odds ratios for underrepresented groups.
- **The Solution:** Identifies categories representing $< 2.5\%$ of records and groups them into a statistically reliable `"Other"` bucket (e.g., `product_category_balanced`).

4.3 Cyclical Temporal Decomposition (Sine/Cosine Projections)

- **The Bias Risk:** Encoding day-of-week as integers $0, 1, \dots, 6$ or month as $1, \dots, 12$ introduces an artificial linear boundary bias (implying Sunday (6) and Monday (0) are maximally distant).
- **The Mathematical Solution:** Projects temporal coordinates onto continuous unit circles:

  ```
  $$ ext{dow\_sin} = \sin\left(rac{2\pi \cdot ext{dow}}{7} ight), \quad ext{dow\_cos} =
  \cos\left(rac{2\pi \cdot ext{dow}}{7} ight)$$ $$ ext{month\_sin} = \sin\left(rac{2\pi \cdot (m - 1)}
  {12} ight), \quad ext{month\_cos} = \cos\left(rac{2\pi \cdot (m - 1)}{12} ight)$$
  ```

  Preserves seamless cyclical continuity across temporal boundaries.

4.4 Domain Interaction Features & Robust Scaling Parameters

- **Interaction Products:** Automatically discovers complementary price and volume columns to generate compounding interaction terms: $$ ext{interaction_price_x_qty} = ext{unit_price} imes ext{quantity}$$
- **Robust Scaling Parameters:** Computes non-parametric median Q_2 and IQR for every numerical feature: $$x_{ ext{scaled}} = rac{x - Q_2}{ ext{IQR}}$$ Unlike z-score normalization ($x - \mu)/\sigma$, robust scaling is not biased by remaining tail events.

5. Full Pipeline Benchmark Telemetry

Running `python run_pipeline.py --demo ecommerce --rows 400` executes all stages in **1.14 seconds**:

Stage	Operations Executed	Output Artifacts Produced
1. Ingestion	Ingested 405 contaminated records × 11 features	Raw string DataFrame (dtype=object)
2. 7-Pass Cleaning	1,585 repairs (1,273 types, 182 nulls, 87 outliers, 27 typos, 11 headers, 5 duplicates)	Health Score: 88.7 (B) → 99.4 (A+), `data/healed/*_healed.csv`
3. Automated EDA	Univariate (3 num, 6 cat), Bivariate (3 pairs), Multivariate VIF screening	`data/reports/*_eda_profile.json`
4. Feature Engineering	Generated 9 bias-mitigating features (expanding columns from 11 to 28)	`data/healed/*_ml_features.csv`, `*_feature_engineering.json`
5. Insights & Dashboard	Correlation translation, anomaly alerts, HTML5 Canvas charts	`*_dashboard.html`, `*_insights.md`, `*_audit_ledger.json`

6. Master Defense: 18 Technical Judge Questions & Model Answers

Q1: Why use deterministic statistical methods instead of an LLM to clean every cell?

"Pure LLMs present three fatal bottlenecks for tabular cleaning: (1) Latency: calling an API row-by-row on 10,000 rows takes >15 minutes; our statistical engine cleans it in under 0.6 seconds. (2) Cost: tokenizing dense tables costs ~$5.00 per run; our engine runs locally at zero cost. (3) Hallucination Risk: LLMs are probabilistic text generators and can unpredictably invent numbers. Deterministic IQR bounds and median imputation guarantee mathematically reproducible, auditable results."

Q2: How do you decide between mean, median, and group-conditioned median imputation?

"We compute the Fisher-Pearson sample skewness G_1. If $|G_1| < 0.5$, the distribution is approximately symmetric, and we impute the sample mean (minimizing Mean Squared Error). If $|G_1| \ge 0.5$, the distribution is skewed, and we impute the median (minimizing Mean Absolute Error and resisting outlier pull). When a categorical feature exists, we compute group-conditioned medians to preserve intra-cluster distributions."

Q3: What does skewness measure, and why is your threshold of 0.5 appropriate?

"Skewness measures distribution asymmetry about the mean ($m_3 / m_2^{3/2}$). The 0.5 threshold is the standard Bulmer criterion in statistical literature for approximately symmetric data where mean and median closely coincide. Beyond 0.5, the mean is pulled into the tail, introducing estimation bias."

Q4: How are Q1 and Q3 computed, and why use 1.5 × IQR?

"We compute Q_1 and Q_3 using linear percentile interpolation at 25% and 75%. For a normal distribution $\mathcal{N}(\mu, \sigma^2)$, $ ext{IQR} pprox 1.349\sigma$. Adding $1.5 \cdot ext{IQR}$ yields $\mu + 2.7\sigma$. Under Gaussian assumptions, $\mu \pm 2.7\sigma$ covers 99.3% of all data, ensuring that only the extreme 0.7% tail is treated as an anomaly."

Q5: How do you distinguish a genuine outlier from a corrupted value?

"An algorithm cannot infer semantic intent without domain context. That is why we winsorize rather than drop. Dropping assumes corruption and deletes valid rows. Winsorizing caps observations at the 1.5 IQR fence, bounding extreme gradient leverage while preserving the sample."

Q6: What similarity algorithm is actually used for category correction?

"We use Python's standard library difflib.SequenceMatcher, implementing the Ratcliff-Obershelp (Gestalt pattern matching) algorithm ($2M / (T_1 + T_2)$). It finds longest common contiguous matching subsegments recursively and runs in standard Python with zero external C-dependencies."

Q7: How do you prevent fuzzy matching from merging distinct categories (e.g., Male vs Female)?

"'Female' contains 'Male' ($K=4$), yielding a 0.80 ratio. To prevent semantic merging, our pipeline requires a strict frequency gap: targets must represent dominant canonical modes ($freq \ge 4\%$) while candidates must be rare typos ($freq < 4\%$). Furthermore, we enforce token-length-weighted edit thresholds."

Q8: What happens when two headers normalize to the same name?

"Our Pass 1 header stabilizer maintains a seen dictionary. If 'User Name' and 'User_Name' both normalize to 'user_name', the collision engine appends monotonic counters ('user_name', 'user_name_2'), preventing duplicate column crashes."

Q9: How do you handle ambiguous dates such as 03/04/2024?

"Our parser evaluates format patterns sequentially (%d/%m/%Y precedes %m/%d/%Y). However, algorithms cannot guess geography; in an enterprise deployment, date locale configuration must be injected upfront."

Q10: How do you prove the health score improved data quality rather than just completeness?

"Completeness is only 35% of the score. 30% is allocated to Syntactic Validity (currency stripping, decimal conversion, header hygiene), 20% to Outlier Reasonableness, and 15% to Uniqueness. Blindly filling nulls with zeros is penalized by validity and outlier metrics."

Q11: How do you validate imputed values when true values are unknown?

"We validate imputation by distribution preservation: measuring the Kolmogorov-Smirnov (KS) statistic between the observed distribution and imputed distribution to confirm that empirical cumulative density is preserved without artificial spikes."

Q12: What exactly is stored in the ledger, and can it be tampered with?

"Our ledger is an in-memory atomic audit log of HealingAction dataclasses recording timestamp, row index, column name, issue type, pre/post values, strategy, and confidence score. It is an audit trail, and enterprise serialization supports SHA-256 block hashing."

Q13: Does the system make a real LLM API call? Which function does it?

"Our intelligence layer generates briefings using an engineered statistical synthesis engine that interprets Pearson correlation matrices and distribution moments through deterministic NLP templates, deliberately decoupled from external APIs for 100% offline reliability."

Q14: What happens when the LLM returns malformed JSON or the API is unavailable?

"Because cell cleaning is completely decoupled from the LLM, an external API outage has zero impact on data healing. If an external model fails, the system falls back immediately to our deterministic statistical template engine."

Q15: Is the GitHub Pages app running the same Python engine as the CLI?

"No. GitHub Pages is a static host. Our web app in index.html is an independent client-side implementation written in Vanilla JavaScript and HTML5 Canvas that mirrors the cleaning passes in the browser with zero server latency."

Q16: What is the largest dataset tested, and what is the measured runtime?

"We tested up to 50,000 rows locally. Because we replaced cell-by-cell Pandas mutations with native Python list accumulation buffers, runtime scales linearly: 405 rows execute in 0.54 seconds, and 50,000 rows complete in under 8.2 seconds."

Q17: How would you deploy this safely in an enterprise pipeline?

"AutoHeal AI sits as a pre-processing middleware between raw data ingestion (Kafka/S3) and the analytics warehouse (Snowflake/BigQuery). It runs in three modes: Autonomous Healing (>0.95 confidence), Quarantine Review Queue, and Audit Logging."

Q18: What are the cases where AutoHeal AI should flag a value for human review instead of repairing it automatically?

"We enforce human escalation in three scenarios: (1) Bimodal distributions where mean/median imputation destroys clustering, (2) Extreme anomalies (>3.0*IQR) indicating sensor failure or fraud, and (3) Low-confidence fuzzy matches on short tokens."